What we actually got was four attribute nodes, one for each attribute set on all of the `project` children. They are inserted into the node list results in the order they are encountered. Just to prove the point, and to demonstrate that (rather handily) `Dom\XPath::query()` returns *references* to node objects inside our `Dom\XMLDocument` object; we can traverse the DOM from the nodes in the list we got back from `query()`:

```
foreach ($results as $node) {
    print "Parent contains : {$node->parentNode->textContent}" . PHP_EOL;
}
Output:
Parent contains : Lovely JS
Parent contains : Lovely PHP
Parent contains : Nasty VB6
Parent contains : Nasty VB6
```

Hopefully you can see that we're actually dealing with three `project` elements, and the "Nasty VB6" appears twice because that element has two attributes. This isn't necessarily tremendously useful, but it lays the groundwork for something that is very useful indeed – *predicates*.

Predicates

The third component of a localization step is zero or more predicates, which are sub-expressions that allow you to further filter the nodes in the context. Here we can see some real power. Remember that `class` and `id` are plastered liberally around most HTML elements because those special attributes are used to drive CSS and JavaScript. We can also make use of them in XPath. Rather than fumbling around the DOM with loops or recursive functions, or needing lots of prior knowledge about a document, you can instead make use of `class` and `id` to write very concise expressions to get right where you need to be.

The fundamental mechanism of CSS is the *selector* expression, which allows styling directives to be applied to elements that match the selector pattern. PHP 8.4 does offer support for DOM traversal using selector expressions, which we will get to after we're done with XPath, but in the meantime, here are a couple of quick examples: `.large div { /*declaration*/ }` would match all `div` elements that are descendants of any element that has `class="large"`, `p.small { /*declaration*/ }` matches all `<p class="small">` elements, `div > p { /*declaration*/ }` would match all `<p>` that